DEVELOPER TOOLS

AI · ML · DS / REVISION NOTES

YOUTUBE

uv — A Faster All-in-One Package Manager

Replacing pip and venv with uv

3
PAGES

Compiled October 2026
github.com/ankitgupta1729/AI-ML-DS

1. UV is a python package manager. It is a single tool that can replace Pip for installing packages, venv or virtual env for creating and managing virtual environments, pip tools for generating lock files and even pipx for installing and running python tools. UV combines all these tools into one. Also, it is a quite bit faster than other options since it is written in RUST. It's created and maintained by Astral (same team behind ruff linter).

2. Installation on Mac using homebrew: `brew install uv`. Then type `uv` in terminal to see all the commands.

3. Now, create a new project directory as `mkdir old_way` and then go to into that directory using `cd old_way`.

Now, in this `old_way` folder, create virtual environment using `python3 -m venv .venv` and then activate it using `source .venv/bin/activate` on Mac. Now, install the packages as `pip install flask requests`. Now create a project file as `touch main.py`. You can see the files and folders using `ls -la`.

Now, to reproduce this project and share with other people, we need to create requirements.txt file using `pip freeze > requirements.txt` and see the content using `cat requirements.txt`.

4. Now, go to one directory up using `cd ..` and create a new project folder for uv as `uv init new_app` and go to this folder using `cd new_app`. If directory is already created then type `uv init` only without mentioning the directory name. It will set up the directory which you are currently using as a new application.

There are some options as well.

`uv init new_app --app` is the default one. It creates a simple project structure for applications, scripts, web servers and things like that.

`uv init new_app --lib` creates a project that is designed to be built and distributed as a python package.

After `uv init new_app`, you will see the following files and folders using `ls -la`.

You can see the readme.md file for documentation. `pyproject.toml` file is a modern way to configure python projects. You can open this project using `code .` in vscode.

You can see the pyproject.toml file where python version is mentioned. Initially dependencies contains empty list.

We don't have to activate virtual environment because uv automatically handles it when we use it first time. To add dependencies, run `uv add flask requests` and it will create the virtual environment at different location which will be mentioned. Also, packages will be installed very quickly. Again see the dependencies in pyproject.toml file. Also, it creates `uv.lock` file and this lock file records exact versions of all the installed dependencies and its sub-dependencies. This ensures that your environment is perfectly reproducible and it does this without creating and activating the virtual environment.

To see the dependencies and sub-dependencies in tree form, run `uv tree` in terminal.

5. Now, you can run the python file in terminal as `uv run main.py`.

You can delete the virtual environment using `rm -rf .venv`. Now, you can see using `ls -la`. Now, if we run again `uv run main.py`, it will create the virtual environment and install libraries very quickly and give the output of main.py. So, using one command, virtual environment is created, activated, install libraries and generate the output and also it is very fast as compared to pip and venv.

Now, if we again delete the virtual environment using `rm -rf .venv` and if we still have uv.lock and pyproject.toml files then we can create virtual environment using `uv sync`. So, if someone has these 2 files on his own machine then we can replicate the virtual environment on our own machine using `uv sync`.

If we want to remove dependency then we can run `uv remove flask` and it will remove that dependency and update the pyproject.toml and uv.lock files.

So, it is very easy to use uv package manager with very few commands and it is faster also.

6. One of the biggest advantage of uv is its smart global caching system. It means that if we have 10 different projects that all use the same version of flask, then uv only stores that flask version and its dependencies once on our disk which saves a lot of space. So, your environments still remains completely isolated from each other. So, there is no risk of conflicts here but you get disk space savings and major speed improvements when installing packages that used before. With pip and venv, we create different virtual environments for different projects, might contain duplicate copies of the same packages. So you not save only disk space but you also save download times and what used to take minutes with pip, can be instantaneous with uv.
7. uv provides `uv pip` subcommand that acts as a direct faster replacement for pip itself.
8. If we want to add packages from requirements.txt file then we can run `uv add -r requirements.txt`. Now, check pyproject.toml file.
9. Another cool feature of uv is that it can replace pipx for installing python based command line tools.

For example, to install ruff tool, we can run `uv tool install ruff` and it can be accessed by anywhere on the system. Run `which ruff` to check the installed location. Run `ruff check` for the current project.

To remove the tool, run `uv tool uninstall ruff` and then check location using `which ruff`.

If you don't want to install ruff permanently, then run `uv tool run ruff check` but when I search using `which ruff`, I will not find it. So, these commands installs it in a temporary environment and then remove/clean it afterwards.

`uv tool run ruff check` is similar to `uvx ruff check` and then use `which ruff` and it will not show location.

10. `uv tool list` command shows all the installed tools. Upgrade all the tools using `uv tool upgrade --all`.